KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

DIGITAL WALLET AND MANAGEMENT SYSTEM

Submitted by

HARISH KUMAR I

25BIT113

Under the Guidance of

MUGILAN R

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled “DIGITAL WALLET AND MANAGEMENT SYSTEM” submitted in partial fulfillment of the requirements for the course **Java Programming** is a **record of my original work** carried out under the guidance of **MUGILAN R**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

(HARISH KUMAR I)

CERTIFICATE

This is to certify that the project entitled **"DIGITAL WALLET AND MANAGEMENT SYSTEM"** is a **bona fide work** carried out by **VI HARISH KUMAR I B.Sc.IT 'B' (2025-2028)**, **INFORMATION TECHNOLOGY**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[Course Name]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place:COIMBATORE

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Digital Wallet and Management System is a Java-based application designed to manage digital financial transactions efficiently. In many situations, handling cash manually can be inconvenient, time-consuming, and prone to errors.

This system provides a digital platform where users can store money, perform transactions such as adding funds and sending money, and maintain a record of all transactions.

The application is developed using Core Java concepts such as classes, objects, methods, loops, and exception handling. It can be implemented as a console-based application or extended into a graphical user interface using Java Swing.

The main objective of this system is to provide a simple, secure, and efficient way to manage digital payments and track financial activities.

1. INTRODUCTION

In today's digital era, cashless transactions are becoming increasingly popular. Digital wallets provide a convenient way for users to store money electronically and perform transactions without the need for physical cash.

Traditional cash handling methods often lead to problems such as loss of money, lack of transaction records, and inconvenience in managing finances. A digital wallet system helps overcome these issues by providing a secure and organized platform for financial management.

The Digital Wallet and Management System is designed to simulate a basic wallet application where users can add money, send money, check balance, and view transaction history.

This project is developed using Core Java programming concepts and demonstrates how Java can be used to build simple financial applications.

1.1 Objective

The main objective of the Digital Wallet and Management System is to develop a Java-based application that simplifies digital transactions and wallet management.

Specific objectives:

- To design a digital wallet system using Core Java
- To enable users to add and manage wallet balance
- To perform secure transactions such as sending money
- To maintain transaction history
- To reduce dependency on physical cash

1.2 Overview

The Digital Wallet and Management System is a menu-driven application developed using Java. It provides users with options to manage their digital wallet efficiently.

Users can add money to their wallet, send money, check their current balance, and view transaction history. The system automatically updates the wallet balance after each transaction and records all activities for future reference.

This system simplifies financial operations and ensures better tracking of transactions.

1.3 Features

Wallet Balance Management – Add and store money digitally

- Money Transfer – Send money easily
- Balance Checking – View current wallet balance
- Transaction History – Record all transactions
- Automatic Updates – Updates balance after each transaction
- User-Friendly Interface – Simple menu-driven system
- Fast Processing – Quick execution of transactions

2.PROGRAM ANALYSIS

Program analysis is an important phase in software development where the problem is studied in detail and the requirements of the system are identified. This stage helps in understanding how the system should work and what features are required to solve the problem efficiently.

The analysis phase focuses on identifying the current issues in the existing system and proposing a computerized solution that improves efficiency and accuracy.

2.1 Problem Definition

Manual financial transactions have several drawbacks:

- Difficulty in tracking expenses
- Risk of losing cash
- No proper transaction history
- Time-consuming process

Therefore, a digital wallet system is needed to provide a secure and efficient way of managing transactions

2.2 Project Overview

The Digital Wallet and Management System allows users to perform basic financial operations digitally.

Main functionalities include:

- Adding money to wallet

- Sending money
- Checking balance
- Viewing transaction history

The system ensures accurate calculations and proper record maintenance.

2.3 Scopes and Limitations

Scope:

- Personal financial management
- Small-scale digital transactions
- Educational demonstration of Java programming

Limitations:

- No real bank integration
- Basic security (no encryption)
- Console-based interface

3.SYSTEM DESIGN

System design is the phase where the structure and components of the system are planned before implementation. It defines how the system will work, how data will flow, and how different modules interact with each other.

The Digital Wallet and Management System is designed as a **menu-driven application** that allows users to perform financial operations easily and efficiently.

The system is divided into multiple modules, each responsible for a specific function

3.1 MODULES

1. User Management Module

This module handles user interaction with the system.

Functions:

- Accept user input
- Display menu options
- Guide user through operations

Example:

- Choosing options like Add Money, Send Money, etc.

2. Wallet Management Module

This module is responsible for maintaining the wallet balance.

Functions:

- Add money to wallet
- Store current balance

- Update balance after each transaction

Example:

Balance = Balance + Added Amount

3. Transaction Processing Module

This module handles all money transfer operations.

Functions:

- Send money
- Validate sufficient balance
- Deduct amount from wallet

Example:

```
If (balance >= amount)
  balance = balance - amount
Else
  Show error
```

4. Transaction History Module

This module keeps track of all transactions performed by the user.

Functions:

- Store transaction details
- Display history

- Maintain records

Example Records:

- Added ₹500
- Sent ₹200

5. Balance Display Module

This module shows the current wallet balance.

Functions:

- Retrieve balance
- Display updated value

6. Validation and Error Handling Module

This module ensures the system runs correctly without errors.

Functions:

- Check invalid input
- Prevent negative transactions
- Handle insufficient balance

Example:

- Cannot send money if balance is low

7. Exit Module

This module allows the user to safely terminate the application.

3.2 SYSTEM ARCHITECTURE

The system follows a simple architecture:

User → Menu → Operation → Processing → Output

Flow Explanation:

1. User selects an option
2. System processes request
3. Balance is updated
4. Result is displayed

3.3 DATA FLOW DESCRIPTION

The data flow in the system is simple and efficient.

Steps:

- User inputs data (amount, choice)
- System processes input
- Data is stored/updated
- Output is displayed

3.4 INPUT DESIGN

Input design focuses on how user data is entered into the system.

Inputs include:

- Menu choice
- Amount to add
- Amount to send

Design Considerations:

- Simple input format

- Error-free entry
- Validation of values

3.5 OUTPUT DESIGN

Output design defines how results are displayed to the user.

Outputs include:

- Current balance
- Transaction success/failure message
- Transaction history

Example Output:

Balance: ₹300

Transaction Successful!

3.6 DATABASE / DATA STORAGE (BASIC)

Since this is a Core Java project, data is stored temporarily.

Storage Used:

- Variables → for balance
- ArrayList → for transaction history

Note:

- Data is not stored permanently

- Can be upgraded using MySQL in future

3.7 SYSTEM WORKING (STEP-BY-STEP)

1. User starts the application
2. Menu is displayed
3. User selects an option
4. System performs operation:
    - Add money
    - Send money
    - Check balance
    - View history
5. System updates data
6. Output is displayed
7. Process repeats until exit

4.METHODOLOGY

Methodology describes the step-by-step process followed to design, develop, and implement the Digital Wallet and Management System. It ensures that the system is developed in a structured and systematic way.

A **sequential development approach** is used in this project, where each phase is completed before moving to the next phase. This helps in better understanding, proper implementation, and error-free execution of the system.

The methodology includes the following stages:

4.1 PROBLEM ANALYSIS

In this phase, the existing system and its problems are studied carefully.

Problems Identified:

- Manual cash handling is risky and inconvenient
- No proper record of transactions
- Difficulty in tracking expenses
- Chances of human errors
- Time-consuming financial management

Solution:

To develop a **Digital Wallet System** that:

- Stores money digitally
- Tracks transactions automatically

- Provides accurate and fast processing

4.2 REQUIREMENT ANALYSIS

This phase identifies what the system should do.

Functional Requirements:

- Add money to wallet
- Send money
- Check balance
- View transaction history

Non-Functional Requirements:

- Easy to use interface
- Fast processing
- Reliable performance
- Basic validation for inputs

4.3 SYSTEM DESIGN

In this phase, the structure of the system is planned.

Design Includes:

- Dividing system into modules
- Designing program flow

- Planning user interaction

Modules Designed:

- Wallet Module
- Transaction Module
- History Module
- Balance Module

This step ensures smooth development and easy maintenance.

4.4 PROGRAM DEVELOPMENT

This is the implementation phase where the actual coding is done.

Technologies Used:

- Core Java
- Scanner class for input
- ArrayList for storing transaction history

Concepts Used:

- Classes and Objects
- Loops (do-while)
- Conditional Statements (if-else, switch)
- Methods

Development Steps:

- Create main class
- Implement menu-driven system
- Write logic for wallet operations
- Handle user inputs

4.5 TESTING AND VALIDATION

Testing ensures that the system works correctly and produces expected results.

Types of Testing:

1. Functional Testing

Checks whether each function works correctly.

2. Input Testing

Checks for valid and invalid inputs.

3. Boundary Testing

Tests edge cases (e.g., zero balance)

Example Test Cases:

Test Case	Input	Expected Output
Add Money	500	Balance = 500
Send Money	200	Balance = 300
Insufficient Balance	1000	Error Message

Test Case	Input	Expected Output
View History	-	Displays records

4.6 IMPLEMENTATION

After testing, the system is implemented successfully.

Implementation Steps:

- Compile Java program
- Execute in console
- Perform operations using menu

The system runs efficiently without errors.

4.7 RESULT AND DOCUMENTATION

In this phase:

- Results are observed and verified
- Screenshots are captured
- Documentation is prepared

Output Includes:

- Updated wallet balance
- Transaction confirmation messages
- Transaction history

4.8 ALGORITHM

Algorithm for Digital Wallet System:

Start

Initialize balance = 0

Display menu

User selects option

If Add Money → Add amount to balance

If Send Money → Check balance → Deduct amount

If Check Balance → Display balance

If History → Display transactions

Repeat until user exits

End

4.9 FLOWCHART (EXPLANATION)

The flowchart represents the logical flow of the system.

Flow:

Start

↓

Display Menu

↓

User Choice

↓

Perform Operation

↓

Update Balance

↓

Display Output

↓

Repeat / Exit

↓

En

5.SOURCE CODE

```
import java.util.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class DigitalWallet {

    static class Transaction {
        enum Type { DEPOSIT, WITHDRAW, TRANSFER_OUT, TRANSFER_IN }

        String id;
        Type   type;
        double amount;
        String description;
        String timestamp;

        Transaction(Type type, double amount, String description) {
            this.id          = "TXN" + System.currentTimeMillis();
            this.type        = type;
            this.amount      = amount;
            this.description = description;
            this.timestamp   = LocalDateTime.now()
                                .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
        }

        @Override
        public String toString() {
```

```
        String sign = (type == Type.DEPOSIT || type == Type.TRANSFER_IN) ? "+" : "-";
        return String.format("  [%s] %-14s %s₹%.2f  |  %s  |  %s",
                id, type, sign, amount, timestamp, description);
    }
}

static class Wallet {
    String          owner;
    String          pin;
    double          balance;
    List<Transaction> history = new ArrayList<>();

    Wallet(String owner, String pin, double initialDeposit) {
        this.owner   = owner;
        this.pin     = pin;
        this.balance = 0;
        if (initialDeposit > 0) deposit(initialDeposit, "Account opened");
    }

    boolean checkPin(String input) { return pin.equals(input); }

    void deposit(double amount, String desc) {
        balance += amount;
        history.add(new Transaction(Transaction.Type.DEPOSIT, amount, desc));
```

```
    }

    boolean withdraw(double amount, String desc) {
        if (amount > balance) return false;
        balance -= amount;
        history.add(new Transaction(Transaction.Type.WITHDRAW, amount, desc));
        return true;
    }

    void printStatement() {
                                                                System.out.println("\n
═══════════════════════════════════════════════════════════════════════
═══════════");
        System.out.println("  Account Statement — " + owner);
                                                                System.out.println("
═══════════════════════════════════════════════════════════════════════
═══════════");
        if (history.isEmpty()) {
            System.out.println("  No transactions yet.");
        } else {
            history.forEach(System.out::println);
        }
        System.out.printf("%n  Current Balance : ₹%.2f%n", balance);
                                                                System.out.println("
═══════════════════════════════════════════════════════════════════════
═══════════");
```

```
    }
}

static Map<String, Wallet> wallets = new HashMap<>();
static Scanner             sc      = new Scanner(System.in);
static Wallet              current = null;

                                static          void          line()                  {          System.out.println("
──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────"); }
static void blank() { System.out.println(); }

static String prompt(String msg) {
    System.out.print("  " + msg);
    return sc.nextLine().trim();
}

static double promptAmount(String msg) {
    while (true) {
        try {
            double v = Double.parseDouble(prompt(msg));
            if (v <= 0) throw new NumberFormatException();
            return v;
        } catch (NumberFormatException e) {
            System.out.println("  ⚠  Please enter a valid positive amount.");
        }
```

```
        }
    }

    static void registerMenu() {
        blank();
        System.out.println("  ╔═══════════════════════════════════════════════════════╗");
        System.out.println("  ║        CREATE NEW WALLET          ║");
        System.out.println("  ╚═══════════════════════════════════════════════════════╝");
        String name = prompt("Full name         : ");
        if (wallets.containsKey(name.toLowerCase())) {
            System.out.println("  ✘ An account with that name already exists.");
            return;
        }
        String pin  = prompt("Set 4-digit PIN   : ");
        if (!pin.matches("\\d{4}")) { System.out.println("  ✘ PIN must be exactly 4 digits."); return; }
        String pin2 = prompt("Confirm PIN       : ");
        if (!pin.equals(pin2))      { System.out.println("  ✘ PINs do not match."); return; }

        double opening = 0;
        String dep = prompt("Opening deposit (₹) [0 to skip]: ");
        try { opening = Math.max(0, Double.parseDouble(dep)); } catch (Exception ignored) {}
```

```
        wallets.put(name.toLowerCase(), new Wallet(name, pin, opening));
        System.out.println("\n ✔ Wallet created successfully for " + name + "!");
    }

    static void loginMenu() {
        blank();
        System.out.println("  ╔═══════════════════════════════════════════════════════════╗");
        System.out.println("  ║                    LOGIN                                  ║");
        System.out.println("  ╚═══════════════════════════════════════════════════════════╝");
        String name = prompt("Name  : ");
        Wallet w = wallets.get(name.toLowerCase());
        if (w == null) { System.out.println("  ✘ Wallet not found."); return; }

        for (int attempt = 3; attempt > 0; attempt--) {
            String pin = prompt("PIN   : ");
            if (w.checkPin(pin)) { current = w; System.out.println("\n ✔ Welcome, " + w.owner
+ "!"); return; }
            System.out.println("  ✘ Wrong PIN. " + (attempt - 1) + " attempt(s) left.");
        }
        System.out.println("  ✘ Too many wrong attempts. Access denied.");
    }

    static void depositMenu() {
        blank();
        System.out.println("  ── DEPOSIT ──");
```

```
        double amount = promptAmount("Amount (₹) : ");
        String note   = prompt("Note       : ");
        current.deposit(amount, note.isEmpty() ? "Deposit" : note);
            System.out.printf("  ✔  ₹%.2f deposited. New balance: ₹%.2f%n", amount,
current.balance);
    }

    static void withdrawMenu() {
        blank();
        System.out.println("  ── WITHDRAW ──");
        System.out.printf("  Available : ₹%.2f%n", current.balance);
        double amount = promptAmount("Amount (₹) : ");
        String note   = prompt("Note       : ");
        if (current.withdraw(amount, note.isEmpty() ? "Withdrawal" : note)) {
              System.out.printf("  ✔  ₹%.2f withdrawn. New balance: ₹%.2f%n", amount,
current.balance);
        } else {
            System.out.println("  ✘  Insufficient balance.");
        }
    }

    static void transferMenu() {
        blank();
        System.out.println("  ── TRANSFER ──");
        String recipient = prompt("Recipient name : ");
```

```
        Wallet target = wallets.get(recipient.toLowerCase());
        if (target == null || target == current) {
            System.out.println("  ✘  Recipient wallet not found (or same account).");
            return;
        }
        System.out.printf("  Available : ₹%.2f%n", current.balance);
        double amount = promptAmount("Amount (₹)     : ");
        String note   = prompt("Note           : ");

        if (!current.withdraw(amount, "Transfer to " + target.owner + (note.isEmpty() ? "" : " – " + note))) {
            System.out.println("  ✘  Insufficient balance.");
            return;
        }
        target.deposit(amount, "Transfer from " + current.owner + (note.isEmpty() ? "" : " – " + note));
        target.history.get(target.history.size() - 1).type = Transaction.Type.TRANSFER_IN;
                                    current.history.get(current.history.size()  -  1).type  =  Transaction.Type.TRANSFER_OUT;

        System.out.printf("  ✔  ₹%.2f transferred to %s. Your balance: ₹%.2f%n",
                amount, target.owner, current.balance);
    }

    static void balanceMenu() {
```

```
        blank();
        System.out.printf("  Account  : %s%n", current.owner);
        System.out.printf("  Balance  : ₹%.2f%n", current.balance);
    }

    static void changePinMenu() {
        blank();
        System.out.println("  ── CHANGE PIN ──");
        String old = prompt("Current PIN   : ");
        if (!current.checkPin(old)) { System.out.println("  ✘ Incorrect PIN."); return; }
        String newPin = prompt("New 4-digit PIN: ");
            if (!newPin.matches("\\d{4}")) { System.out.println("  ✘ PIN must be 4 digits.");
return; }
        String confirm = prompt("Confirm new PIN: ");
        if (!newPin.equals(confirm))  { System.out.println("  ✘ PINs do not match.");   return; }
        current.pin = newPin;
        System.out.println("  ✔ PIN changed successfully.");
    }

    static void walletMenu() {
        while (current != null) {
            blank();
                                                                System.out.println("
╔═══════════════════════════════════════════════════════════╗");
            System.out.printf ("  ╟   %-28s╢%n", current.owner);
```

```
        System.out.printf ("  ╟   Balance: ₹%-19.2f╢%n", current.balance);
                                                                    System.out.println("
╟──────────────────────────────────────────────────────╢");
        System.out.println("  ╟  1. Deposit                  ╢");
        System.out.println("  ╟  2. Withdraw                 ╢");
        System.out.println("  ╟  3. Transfer                 ╢");
        System.out.println("  ╟  4. View Statement           ╢");
        System.out.println("  ╟  5. Check Balance            ╢");
        System.out.println("  ╟  6. Change PIN               ╢");
        System.out.println("  ╟  7. Logout                   ╢");
                                                                    System.out.println("
╙──────────────────────────────────────────────────────╜");
        String choice = prompt("Choose (1-7): ");
        switch (choice) {
            case "1" -> depositMenu();
            case "2" -> withdrawMenu();
            case "3" -> transferMenu();
            case "4" -> current.printStatement();
            case "5" -> balanceMenu();
            case "6" -> changePinMenu();
            case "7" -> { System.out.println("  ✔  Logged out."); current = null; }
            default  -> System.out.println("  ⚠  Invalid option.");
        }
    }
}
```

```
  static void mainMenu() {
                                                                System.out.println("\n
╔═══════════════
```

```
        wallets.put("demo user", demo);


        mainMenu();
    }
}
```

6. INPUT AND OUTPUT SCREENS

Main inputs:

1. **Menu choices**
    - 1 → Register
    - 2 → Login
    - 3 → Exit
2. **During Registration**
    - Name
    - 4-digit PIN
    - Confirm PIN
    - Opening deposit (optional)
3. **During Login**
    - Name
    - PIN
4. **After Login (Wallet Menu)**
    - Deposit → amount + note
    - Withdraw → amount + note
    - Transfer → recipient + amount + note
    - Change PIN → old PIN + new PIN

output

1. Main Menu

```
╔═══════════════════════════════════════════════════════╗
║      DIGITAL WALLET SYSTEM      ║
╚═══════════════════════════════════════════════════════╝
```

1. Register

2. Login

3. Exit

Choose (1-3):

2. Registration Success

✔ Wallet created successfully for Arun!

3. Login Success / Failure

✔ Welcome, Arun!

or

✘ Wrong PIN. 2 attempt(s) left.

4. Deposit Output

── DEPOSIT ──

Amount (₹) : 1000

✔ ₹1000.00 deposited. New balance: ₹6000.00

5. Withdraw Output

── WITHDRAW ──

Available : ₹6000.00

✔ ₹500.00 withdrawn. New balance: ₹5500.00

6. Transfer Output

── TRANSFER ──

Recipient name : Ravi

✔ ₹1000.00 transferred to Ravi. Your balance: ₹4500.00

7. Statement Output

═══

Account Statement — Arun

═══

[TXN...] DEPOSIT +₹1000.00 | 2026-04-09 10:30:12 | Deposit

[TXN...] WITHDRAW -₹500.00 | 2026-04-09 10:35:10 | Withdrawal

Current Balance : ₹5500.00

═══

8. Logout / Exit

✔ Logged out.

or

Goodbye

7.DISCUSSION & CONCLUSION

7.1 Discussion

The Digital Wallet and Management System was developed to provide a simple and efficient solution for handling digital financial transactions. The system focuses on reducing the complexity of managing money manually and offers a structured way to perform wallet operations.

During the development of this project, various Core Java concepts such as classes, objects, loops, conditional statements, and data structures like ArrayList were successfully implemented. These concepts helped in designing a menu-driven application that performs all required operations effectively.

The system allows users to:

- Add money to the wallet
- Send money securely (within available balance)
- Check current balance instantly
- View complete transaction history

Each module in the system works together to ensure smooth operation. The wallet management module maintains accurate balance updates, while the transaction module ensures that no invalid transactions occur, such as sending money beyond the available balance.

Testing was conducted using different inputs to verify system performance. The results showed that:

- The system provides accurate calculations
- Transactions are processed quickly

- Errors such as insufficient balance are handled properly

The system successfully eliminates the need for manual record-keeping by maintaining a digital transaction history. This improves transparency and helps users track their financial activities easily.

Advantages of the System

- Reduces manual effort in managing money
- Provides quick and accurate transaction processing
- Maintains proper transaction records
- Easy to use and understand
- Improves efficiency in financial management

Limitations of the System

- Does not support real-time banking integration
- Lacks advanced security features (like encryption or authentication)
- Data is not stored permanently (no database used)
- Limited to console-based interface

Future Enhancements

The system can be further improved by adding:

- User authentication (PIN or password)

- Graphical User Interface (Java Swing)
- Database integration (MySQL) for permanent storage
- Mobile app version
- Online payment integration

7.2 CONCLUSION

The Digital Wallet and Management System has been successfully developed using Core Java to demonstrate how digital transactions can be managed efficiently through a software application.

The system achieves its main objectives by providing functionalities such as adding money, sending money, checking balance, and maintaining transaction history. It simplifies financial management and reduces dependency on manual cash handling methods.

Through this project, practical knowledge of Java programming has been applied to solve real-world problems. The use of modular design and structured programming techniques ensures that the system is easy to understand, maintain, and extend.

Although the current system is basic, it lays a strong foundation for developing more advanced financial applications. With future enhancements such as security features and database connectivity, the system can be transformed into a fully functional digital wallet solution.

In conclusion, this project demonstrates the importance of digital systems in modern financial management and highlights the capability of Java in developing efficient and reliable applications.

8.REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com